ORGNZ-07: Advanced Permission Patterns

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 7 of 10 | **Created:** January 2026 | **Last Updated:** 10/14/2026

Overview

This notebook covers advanced permission patterns including record-level permissions, field-based access, and combining multiple access control mechanisms for enterprise-scale data governance.

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	Account admin or IAM policy management access
Knowledge	Completed ORGNZ-06 (Security Context)
Data	At least 1 hour of log data

Table of Contents

1. Record-Level Permissions
2. Record-Level Policy Examples
3. Field-Level Access
4. Combined Permission Patterns
5. Policy Boundaries
6. Enterprise Architecture Patterns
7. Testing Permissions
8. Best Practices

Learning Objectives

By the end of this notebook, you will:

- Implement record-level permissions using various attributes
- Understand field-level access restrictions
- Use policy boundaries for reusable condition management
- Combine bucket, record, and field-level policies
- Design enterprise-scale permission architectures

Record-Level Permissions

Record-level permissions filter data at query time based on record attributes:



Supported Record-Level Conditions

Condition	Description	Example	Tables it applies to
`storage:k8s.namespace.name`	Kubernetes namespace	`= 'production'`	events, security.events, bizevents, logs, metrics, spans, smartscape
`storage:k8s.cluster.name`	Kubernetes cluster	`= 'main-cluster'`	events, security.events, bizevents, logs, metrics, spans, smartscape
`storage:host.name`	Host name	`= 'web-server-01'`	events, security.events, bizevents, logs, metrics, spans, smartscape
`storage:dt.host_group.id`	Host group	`STARTSWITH 'prod-'`	events, security.events, bizevents, logs, metrics, spans, smartscape
`storage:aws.account.id`	AWS account	`= '123456789012'`	events, security.events, bizevents, logs, metrics, spans, smartscape
`storage:gcp.project.id`	GCP project	`= 'my-project'`	events, security.events, bizevents, logs, metrics, spans, smartscape

Condition	Description	Example	Tables it applies to
`storage:azure.subscription`	Azure subscription	`= 'sub-id'`	events, security.events, bizevents, logs, metrics, spans, smartscape
`storage:azure.resource.group`	Azure resource group	`= 'my-rg'`	events, security.events, bizevents, logs, metrics, spans, smartscape
`storage:dt.security_context`	Custom context	`MATCH ('team-*')`	events, security.events, bizevents, system, logs, metrics, spans, entities, smartscape, user.events, user.sessions
`storage:log.source`	Log source	`= '/var/log/audit/audit.log'`	logs
`storage:metric.key`	Metric key	`STARTSWITH 'dt.host.'`	metrics
`storage:event.kind`	Event kind	`= 'DAVIS_PROBLEM'`	events, security.events, bizevents, system
`storage:event.type`	Event type	`= 'CUSTOM_INFO'`	events, security.events, bizevents, system
`storage:event.provider`	Event provider	`= 'my-app'`	events, security.events, bizevents, system

Condition	Description	Example	Tables it applies to
`storage:frontend.name`	Frontend (RUM application) name	`= 'www.example.com'`	user.events, user.sessions, metrics, smartscape

A condition only restricts the tables listed for it. `storage:log.source` is the natural condition for audit-log access — it scopes `storage:logs:read` to named log sources without a dedicated bucket.

> **Sources:** Permissions in Grail (DT docs) — supported record-level fields and the tables each applies to, read 09/24/2026.

Record-Level Policy Examples

Kubernetes Namespace Isolation

```
{
  "name": "production-namespace-access",
  "description": "Access only to production namespace data",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH
'default_'; ALLOW storage:logs:read WHERE storage:k8s.namespace.name = 'production';"
}
```

Host Group Based Access

```
{
  "name": "web-tier-access",
  "description": "Access to web tier hosts only",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH
'default_'; ALLOW storage:logs:read, storage:metrics:read WHERE
storage:dt.host_group.id STARTSWITH 'web-';"
}
```

Combined Namespace and Host Group

```
{
  "name": "prod-web-tier-access",
  "description": "Production web tier only",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH
'default_'; ALLOW storage:logs:read WHERE storage:k8s.namespace.name = 'production' AND
storage:dt.host_group.id STARTSWITH 'web-';"
}
```

Cloud Account Isolation

```
{
  "name": "aws-team-account-access",
  "description": "Access to specific AWS account",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH
'default_'; ALLOW storage:logs:read, storage:metrics:read, storage:spans:read WHERE
storage:aws.account.id = '123456789012';"
}
```

Field-Level Access

Field-level access in Grail uses **fieldsets**, not per-field permissions. A fieldset is a named group of sensitive fields, and reading those fields requires `storage:fieldsets:read` on that fieldset. A user without it does not get masked values — the fields are left out: *"If you don't have sufficient permissions, sensitive fields won't be shown in the result."*

> **Corrected 09/24/2026.** Earlier versions of this notebook showed `DENY storage:logs:read:user.email` statements. There is no per-field permission suffix, and a policy written that way will not validate.

Fieldset	Covers
`builtin-sensitive-spans`	Span fields considered sensitive
`builtin-request-attributes-spans`	Span fields holding request-attribute data marked sensitive
`builtin-sensitive-user-events-and-sessions`	Sensitive fields in `user.events` and `user.sessions`
Custom fieldset	Fields you name, scoped to buckets or tables

"The predefined fieldsets apply to spans, user.events and user.sessions only. They don't apply to logs or events." For log fields such as `user.email`, define a custom fieldset: *"You can define your custom fieldsets, and to which scope they apply (either buckets, or tables, otherwise all buckets and tables)."*

Fieldset Policy Example

```
{
  "name": "sensitive-span-fields",
  "description": "Allow reading the built-in sensitive span fields",
  "statementQuery": "ALLOW storage:fieldsets:read WHERE storage:fieldset-name =
\"builtin-sensitive-spans\";"
}
```

Grant this only to groups that need the fields; everyone else keeps `storage:spans:read` and does not see them. To mask PII *values* rather than hide whole fields, mask at capture or ingest (OneAgent or OpenPipeline masking) — see **OPLOGS-08**.

> **Sources:** Permissions in Grail (DT docs).

Combined Permission Patterns

Pattern 1: Layered Access Control

Combine bucket and record-level permissions:

```
Layer 1: Bucket access
  ALLOW storage:buckets:read WHERE bucket-name IN ('team_logs', 'shared_logs')

Layer 2: Record filtering
  ALLOW storage:logs:read WHERE k8s.namespace.name = 'team-namespace'

Layer 3: Sensitive fields (optional, only for groups that need them)
  ALLOW storage:fieldsets:read WHERE storage:fieldset-name = "&lt;fieldset-name&gt;"
```

Pattern 2: Multi-Team Shared Bucket

Multiple teams share a bucket with security context isolation:

```
// Team A policy
{
  "name": "team-a-shared-bucket",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name =
'shared_logs'; ALLOW storage:logs:read WHERE storage:dt.security_context MATCH ('team-
a*');"
}

// Team B policy
{
  "name": "team-b-shared-bucket",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name =
'shared_logs'; ALLOW storage:logs:read WHERE storage:dt.security_context MATCH ('team-
b*');"
}
```

Pattern 3: Environment-Based Tiering

```
// Production access (restricted)
{
  "name": "production-access",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH
'prod_'; ALLOW storage:logs:read WHERE storage:dt.security_context = 'env:production';"
}

// Non-production access (broader)
{
  "name": "non-production-access",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH
'default_'; ALLOW storage:logs:read WHERE storage:dt.security_context MATCH
('env:dev*', 'env:staging*', 'env:qa*');"
}
```

Pattern 4: Multi-Dimensional MATCH() for Transversal Teams

When teams need access by component type (database, networking, OS) across multiple applications, encode multiple dimensions into the security context and use MATCH() to slice across application boundaries:

```
// Database team: all DB-layer components across all applications (Grail)
{
  "name": "db-team-grail-access",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH
'default_'; ALLOW storage:logs:read WHERE storage:dt.security_context MATCH
('comp:db*');"
}

// Application team: all component layers for one application (Grail)
{
  "name": "easytrade-team-access",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH
'default_'; ALLOW storage:logs:read WHERE storage:dt.security_context MATCH
('*/app:easytrade');"
}
```

> MATCH() supports `*` wildcards at any position and is **storage domain only**. For Classic entity access (hosts, services), use `startsWith` per component type. The context format is `comp:<component>/bu:<business-unit>/app:<application>` — place the transversal dimension first so `startsWith` cuts correctly. See **IAM-05: Boundary Design** for the full two-domain pattern.

Policy Boundaries

Policy boundaries decouple the "what" (policy) from the "where" (conditions), enabling reusable condition sets that can be applied across multiple policies.

What Are Boundaries?

Aspect	Description
Purpose	Bundle conditions for reuse across multiple policies
Scope	Record-level and resource-level restrictions
Relationship	Always used together with a policy — boundaries alone don't restrict anything
Limit	Maximum 10 restrictions per boundary

How Boundaries Work

While **policies** define *which features and data* users can access, **boundaries** define *where* users can access them:

```
Policy: ALLOW storage:logs:read, storage:metrics:read, storage:spans:read
Boundary: storage:k8s.namespace.name = "production"
          storage:dt.host_group.id STARTSWITH "prod-"
```

When assigned together, the boundary's conditions are added to the permissions they apply to. The Policy boundaries page does not state how two different lines in one boundary combine — it documents that *"Only conditions added to service configuration will be applied to relative permissions"* and that *"When a boundary contains repeated condition names, then each policy statement is multiplied by such conditions."* Check the effective access on a test group before relying on a multi-line boundary. For a guaranteed AND of two conditions, write one policy statement with `AND`, or use policy templating.

Boundary Rules

Rule	Detail
No AND operator	*"Boundaries don't support the AND operator, every line of a boundary can only consist of one condition."*
No logical operators	*"One line per condition, no logical operators allowed."* For complex logic, use policy templating
Reusable	Same boundary can be applied to multiple policies
Max 10 restrictions	Create additional boundaries if more are needed

Creating Boundaries

1. Go to **Account Management** > **Identity & access management** > **Policy management**
2. Select the **Boundaries** tab
3. Select **Create boundary**
4. Enter boundary name and conditions (one per line)
5. Save and assign to group policies

Example: Regional Boundary

```
# EU Region Boundary
storage:bucket-name STARTSWITH "eu_"
storage:azure.subscription = "eu-subscription-id"
```

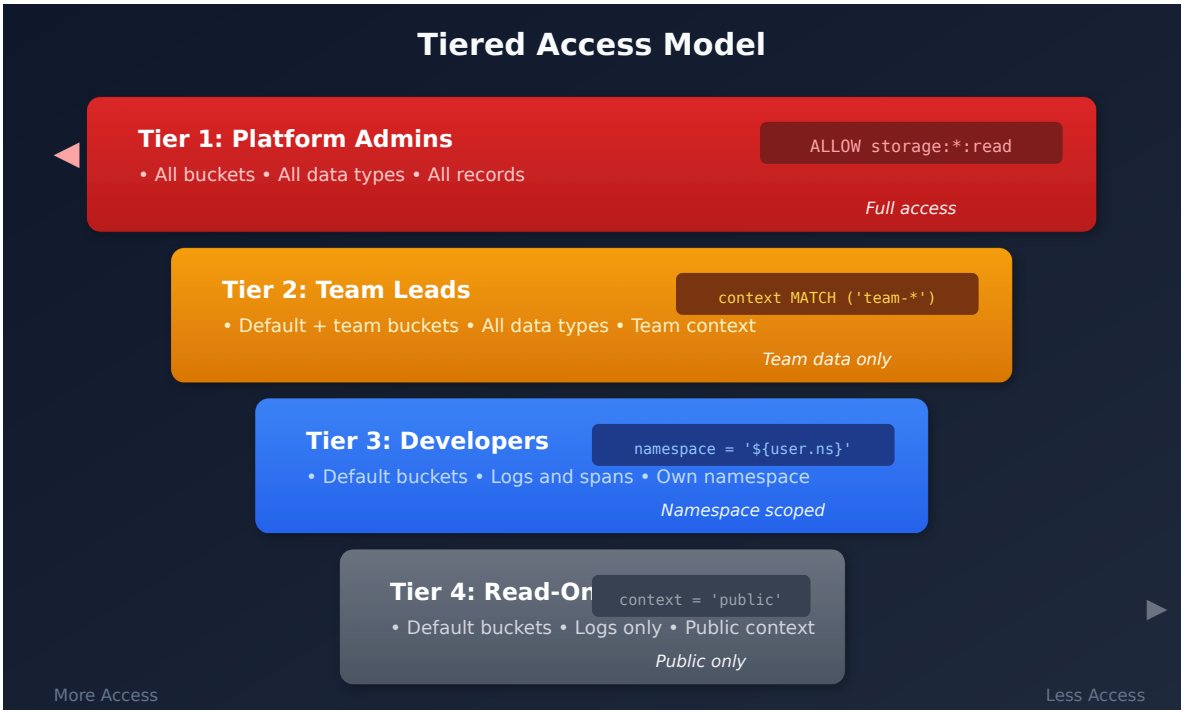
This boundary can then be applied alongside any policy (logs access, metrics access, etc.). Each line restricts only the permissions it applies to, so confirm the effective access on a test group.

Sources: Policy boundaries (DT docs) — all quotes in this section, read 10/14/2026.

Tip: Use boundaries when you have the same set of conditions (e.g., "production only" or "EU region only") that need to apply to multiple policies. This avoids duplicating conditions across every policy definition.

Enterprise Architecture Patterns

Tiered Access Model



Geographic/Regulatory Model

Region	Buckets	Context	Policy
EU	eu_*	region:eu	bucket-name STARTSWITH 'eu_' AND security_context MATCH ('region:eu*')
US	us_*	region:us	bucket-name STARTSWITH 'us_' AND security_context MATCH ('region:us*')

Tip: Use policy boundaries to define regional restrictions once, then apply the same boundary to all team policies within that region.

Testing Permissions

DQL: Access Distribution Audit

Measure coverage of key access control fields across your log data:

```
// Verify accessible data distribution across all key access control dimensions
fetch logs, from:-1h
| summarize
    total = count(),
    withSecurityContext = countIf(isNotNull(dt.security_context)),
    withNamespace = countIf(isNotNull(k8s.namespace.name)),
    withHostGroup = countIf(isNotNull(dt.host_group.id))
```

Best Practices

Practice	Rationale
Start with least privilege	Grant minimum required access
Use groups, not individuals	Easier management, consistent access
Use policy boundaries for reuse	Avoid duplicating conditions across policies
Document all policies	Audit trail and governance
Test policies before deployment	Prevent access issues
Regular access reviews	Remove unnecessary permissions
Use MATCH for array fields	Required for correct evaluation
Combine bucket + record level	Defense in depth

Next Steps

Continue with the ORGNZ series:
- **ORGNZ-08**: Grail Segments

References

- Configure advanced permissions with security context
- Policy boundaries

- IAM policy reference
- Enhance data management with Grail

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.